임베디드 시스템

Embedded Systems

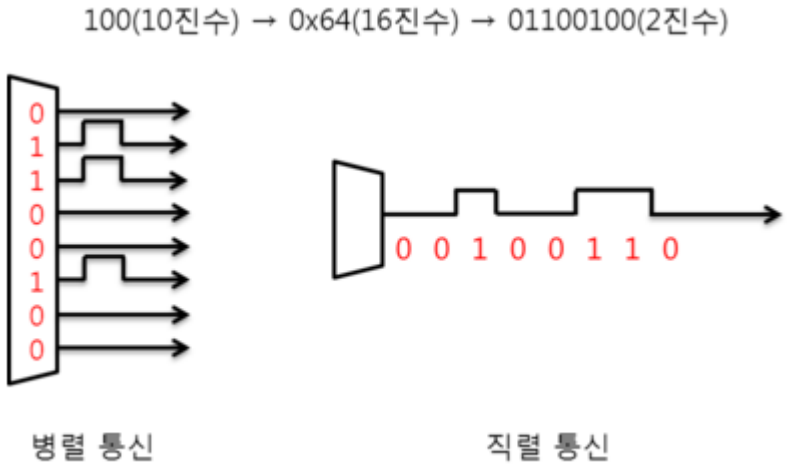
Serial | LCD | FND | IR

강원대학교 컴퓨터공학과 권용훈

Serial(1/3)

직렬(Serial) 통신

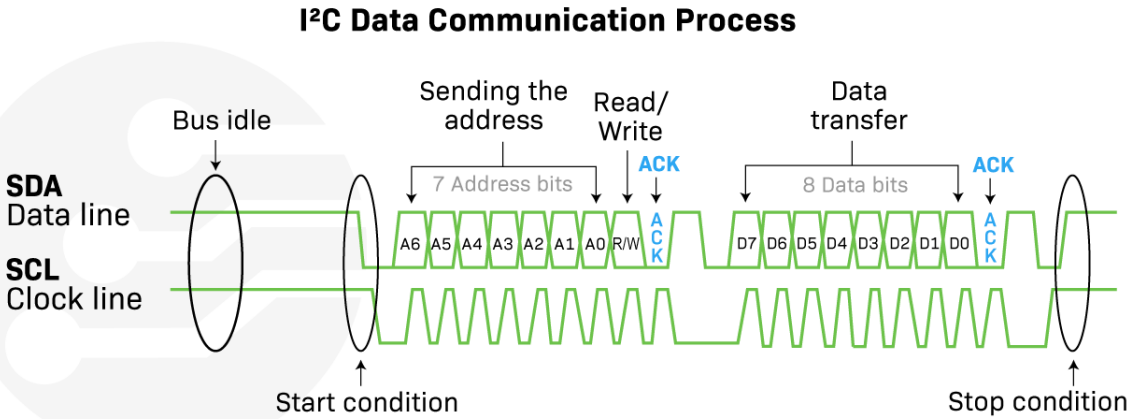
- 하나 혹은 두개의 의 신호선을 이용하여 데이터(1bit)를 주고 받는 통신
- 여러 개의 신호를 보내는 병렬(parallel) 통신의 반대 개념
- 논리 값 1(HIGH)와 0(LOW)으로 구성된 데이터
- 직렬 통신 방식
  - 동기식과 비동기식
  - Single-ended & Differential
  - 전이중 방식과 반이중 방식



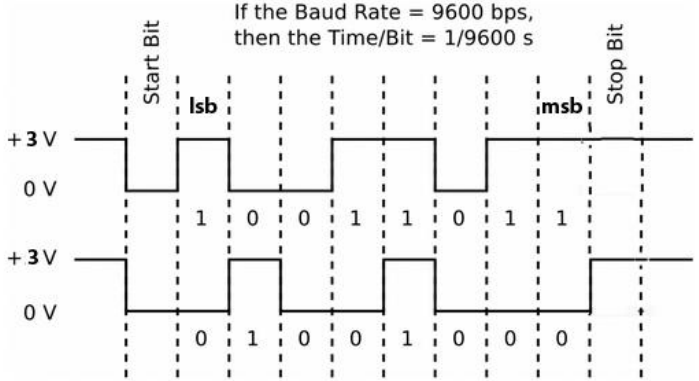
Serial(2/3)

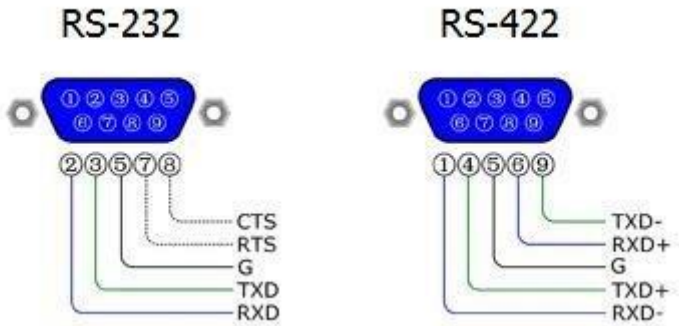
동기식과 비동기식

- 동기식(Synchronous): 클럭 신호를 이용하여 동기화
  - I2C, SPI, I2S



- 비동기식(Asynchronous): 통신 속도를 동일하게 설정하여 통신
  - UART, RS-422, RS-232C, USB

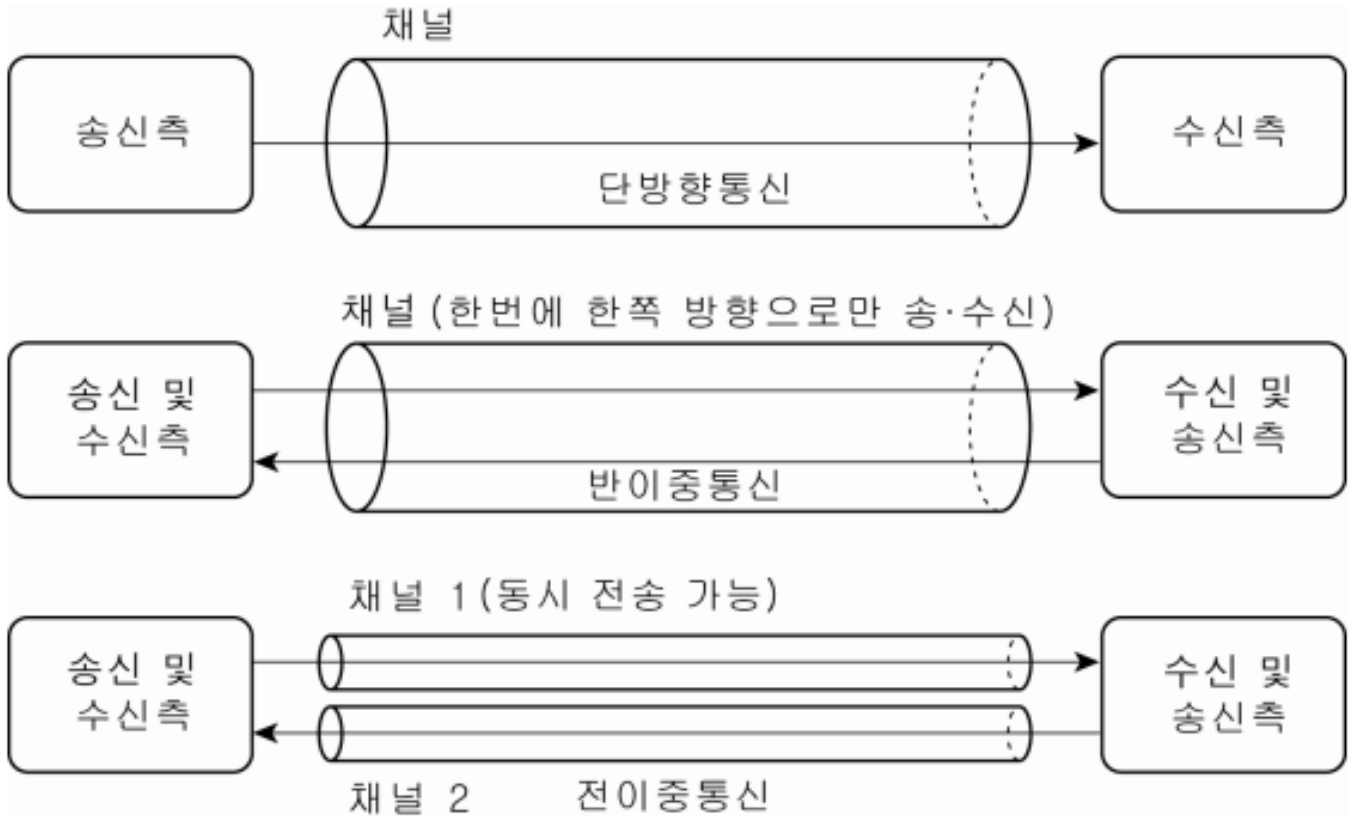




Serial(3/3)

전이중통신과 반이중통신

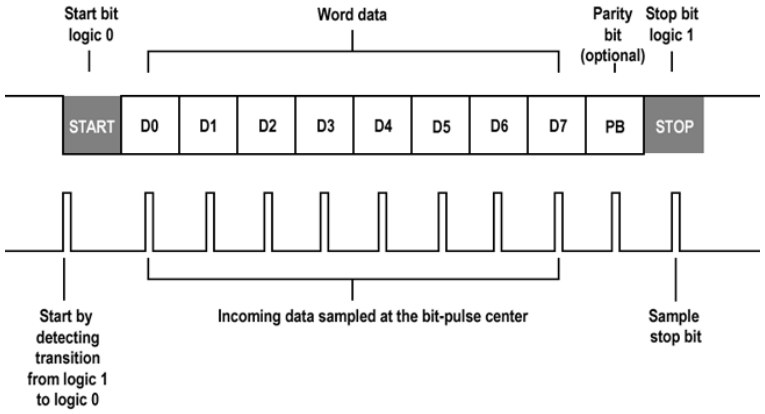
- RX(Reciever)와 TX(Transmitter)의 여부에 따라 구분

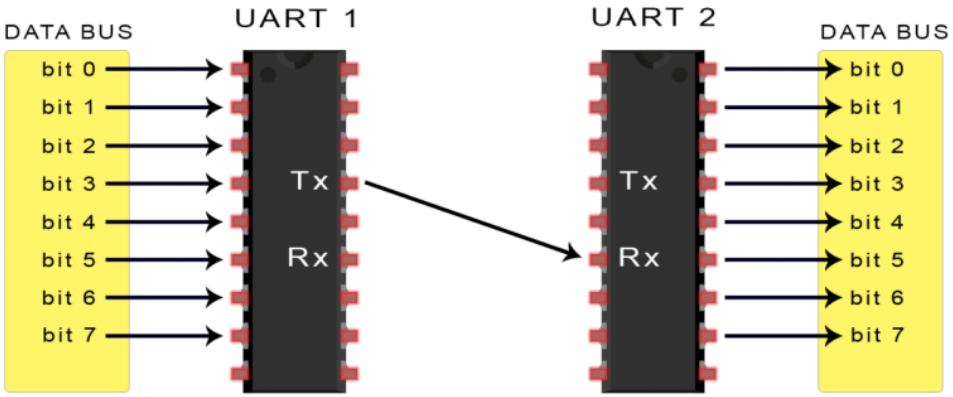


UART

UART(Universal Asynchronous Receiver & Transmitter)

- 병렬 데이터를 직렬 방식으로 전환하여 데이터를 전송하는 방식
- 비동기식의 전이중 직렬 통신이며 클럭 신호없이 baud rate를 이용하여 통신 속도를 조절
- 디버깅용 문자출력(printf), 시스템간의 데이터 교환, AT 등에 활용
- EIA(The Electronic Industry Alliance) 통신 표준: RS-232, RS-422, RS-485

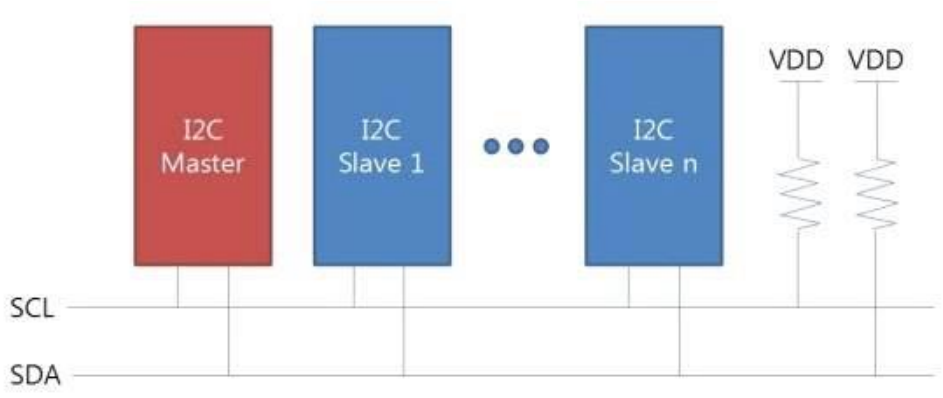




I2C(1/4)

I2C 통신

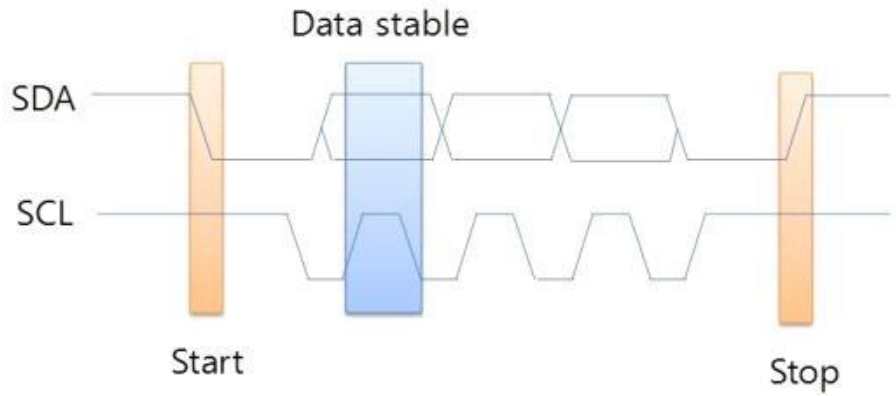
- I2C(Inter-Integrated Circuit): 두 개의 버스를 이용한 직렬 데이터 통신 방식
- Master와 Slave로 구분되며, Master와 Slave 간에 1 : n 연결이 가능하다.



- SCL(Serial Clock): 클럭을 통한 동기화를 위한 핀
- SDA(Serial Data): 데이터를 주고 받기 위한 핀
- Address: slave에 주어지는 고유한 주소 값

I2C(2/4)

I2C 통신 방식



- 두 핀 모두 기본 값은 HIGH 이며, SDA가 LOW 로 떨어질 때 시작 신호로 판단하여 통신이 시작된다.
- 통신할 Slave 주소를 송신하여 해당 주소의 Slave와 통신을 수행
- SCL이 LOW 일 때, 데이터를 비트로 변환한다.
- SCL이 HIGH 일 때, 데이터를 읽어온다.
- Master는 두 핀 모두 HIGH 일 때, SDA를 HIGH로 변경하여 통신을 종료한다.

I2C(3/4)

I2C 통신의 장단점

- 장점
  - 2개의 입출력 핀과 소프트웨어를 이용하여 여러 장치들과 Master & Slave 사이의 반 이중 통신 제어
  - 시스템 동작 중 일 때도 I2C 버스에 추가 제거가 가능하다.
  - SCL 핀으로 클록을 맞추어서 동작하는 동기방식으로 안정적이다.
  - 하드웨어 구성이 간단하며 대화형 동작 가능하다.
  - 멀티 마스터 설정 가능
  - 칩 생산 비용과 설계비용, 기기의 전력 소모 감소
- 단점
  - 최대 100kHz 또는 고속 최대 400kHz까지의 다른 통신에 비교적 저속의 데이터 전송
  - 저속 주변 장치와의 데이터 통신 및 주변 장치의 동작 모드 설정에 적합
  - 동시 양방향 통신 불가
  - 7bit 주소 길이 제한으로 최대 128개 Slave 장치 주소 충돌 가능성 높음

I2C(4/4)

I2C 핀맵

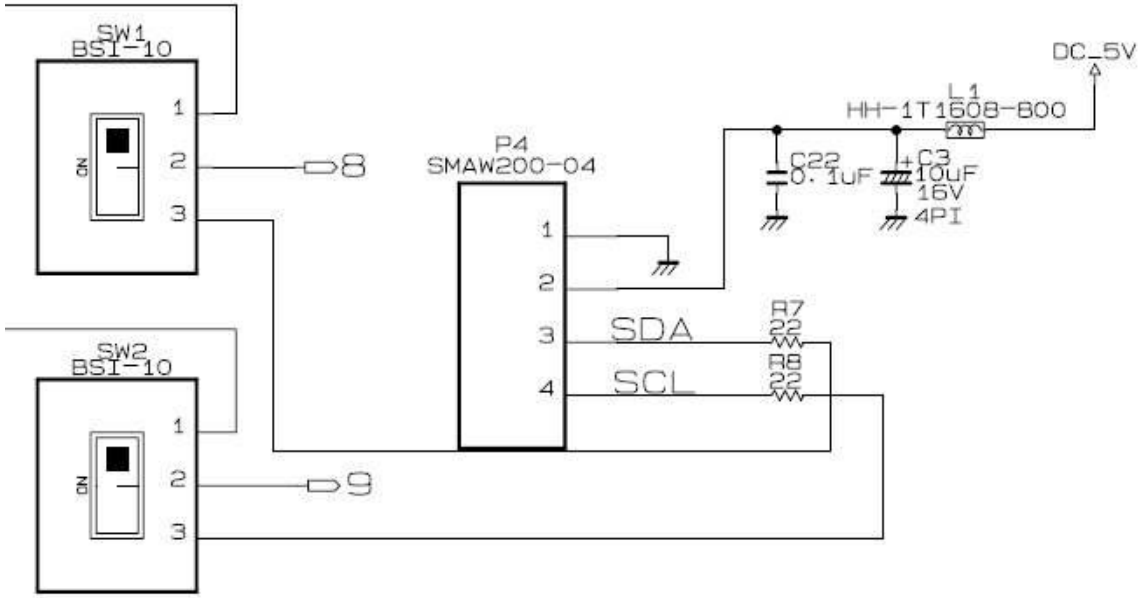
BCM	wPi	Name	Mode	V	Physical	V	Mode	Name	wPi	BCM
		3.3v			1 || 2			5v		
2	8	SDA.1	ALT0	1	3 || 4			5v		
3	9	SCL.1	ALT0	1	5 || 6			0v		
4	7	GPIO. 7	IN	1	7 || 8	0	IN	TxD	15	14
		0v			9 || 10	1	IN	RxD	16	15
17	0	GPIO. 0	IN	1	11 || 12	1	IN	GPIO. 1	1	18
27	2	GPIO. 2	IN	0	13 || 14			0v		
22	3	GPIO. 3	IN	1	15 || 16	0	OUT	GPIO. 4	4	23
		3.3v			17 || 18	0	IN	GPIO. 5	5	24
10	12	MOSI	ALT0	0	19 || 20			0v		
9	13	MISO	ALT0	0	21 || 22	0	IN	GPIO. 6	6	25
11	14	SCLK	ALT0	0	23 || 24	1	OUT	CE0	10	8
		0v			25 || 26	1	OUT	CE1	11	7
0	30	SDA.0	IN	0	27 || 28	0	IN	SCL.0	31	1
5	21	GPIO.21	OUT	0	29 || 30			0v		
6	22	GPIO.22	IN	0	31 || 32	0	IN	GPIO.26	26	12
13	23	GPIO.23	IN	0	33 || 34			0v		
19	24	GPIO.24	IN	0	35 || 36	0	OUT	GPIO.27	27	16
26	25	GPIO.25	IN	0	37 || 38	0	IN	GPIO.28	28	20
		0v			39 || 40	0	IN	GPIO.29	29	21

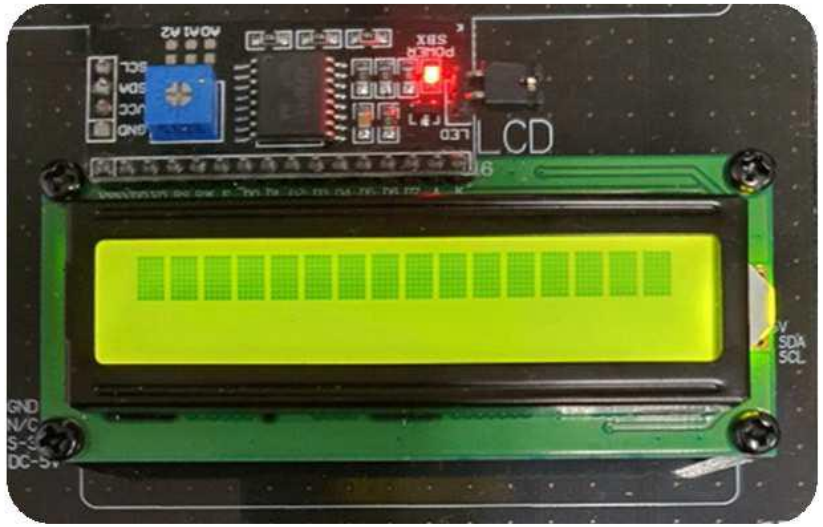
연결핀	기능	Raspberry Pi 핀 배열
SCL0	클럭	SPI_SCLK
SDA0	데이터 송수신 핀	SPI_MOSI
SCL1	클럭	SPI_MISO
SDA1	데이터 송수신 핀	SPI_CEn

LCD(1/8)

LCD (Liquid Crystal Display)

- ASCII 문자를 표현할 수 있는 장치





BCM	wPi	Name	Mode	V	Physical		V	Mode	Name	wPi	BCM
		3.3v			1	2			5v		
2	8	SDA.1	ALT0	1	3	4			5v		
3	9	SCL.1	ALT0	1	5	6			0v		
4	7	GPIO. 7	IN	1	7	8	0	IN	TxD	15	14
		0v			9	10	1	IN	RxD	16	15

LCD(2/8)

Text LCD 모듈의 제어 명령

fosc = 270kHz

기능	제어신호		제어명령어							실행시간
	Rs	R/nW	DB6	DB5	DB4	DB3	DB2	DB1	DB0	(Max.)
Clear Display	0	0	0	0	0	0	0	0	1	1.52ms
Return Home	0	0	0	0	0	0	0	1		1.52ms
Entry Mode Set	0	0	0	0	0	0	1	I/D	S	37us
Display On/Off Control	0	0	0	0	0	1	D	C	B	37us
Cursor or Display Shift	0	0	0	0	1	S/C	R/L			37us
Function Set	0	0	0	1	DL	N	F			37us
Set CG RAM Address	0	0	0	1	CG RAM Address					37us
Set DD RAM Address	0	0	1	DD RAM Address						37us
Read busy flag and address	0	1	BF	Address Count						0us
Data Write to CGRAM or DDRAM	1	0	Write Data							41us
Data Read from CGRAM or DDRAM	1	1	Read Data							41us

(1) Clear Display : 0b0000 0001 (0x01)

전체 화면을 지우고 DDRAM 주소를 0으로 설정한 후 커서를 홈 위치로 옮기면 DDRAM 어드레스 카운트를 0으로 한다.

데이터 버스에는 0000 0001 (0x01)이 전송된다.

Rs	R/nW	DB6	DB5	DB4	DB3	DB2	DB1	DB0
0	0	0	0	0	0	0	0	1

(2) Return Home : 0b0000001x (0x02)

커서를 홈 위치로 옮긴다. DDRAM 주소 카운터를 0으로 하고 디스플레이 시프트된 것을 본래의 위치로 되돌린다.

Rs	R/nW	DB6	DB5	DB4	DB3	DB2	DB1	DB0
0	0	0	0	0	0	0	1	X

(3) Entry Mode Set

MCU가 LCD 모듈에 데이터를 Read/Write할 경우에 DDRAM 어드레스 즉 커서의 위치를 오른쪽으로 증가할 것인지 왼쪽으로 감소할 것인지 결정하며, 화면을 시프트 할 것인지 설정한다.

Rs	R/nW	DB6	DB5	DB4	DB3	DB2	DB1	DB0
0	0	0	0	0	0	1	I/D	S

I/D => Increment/Decrement S => Shift

I/D	S	Value	기능
0	0	0b 0000 0100 (0x04)	커서(어드레스) 좌측 이동, 화면 이동 없음
0	1	0b 0000 0101 (0x05)	커서(어드레스) 좌측 이동, 화면 이동
1	0	0b 0000 0110 (0x06)	커서(어드레스) 우측 이동, 화면 이동 없음
1	1	0b 0000 0111 (0x07)	커서(어드레스) 우측 이동, 화면 이동

LCD(3/8)

(4) Display On/Off

현재 DD RAM의 내용이 화면에 표시되도록 On/Off 설정하거나, 커서(어드레스)를 On/Off하거나, 커서 위치의 문자를 깜빡이게 할 것인지 설정한다.

Rs	R/nW	DB6	DB5	DB4	DB3	DB2	DB1	DB0
0	0	0	0	0	1	D	C	B

D => Display C => Cursor(Address) B => Blink

D	C	V	Value	기능		
				화면	커서	커서 점멸
0	0	0	0b 0000 1000 (0x08)	OFF	OFF	OFF
0	0	1	0b 0000 1001 (0x09)	OFF	OFF	ON
0	1	0	0b 0000 1010 (0x0A)	OFF	ON	OFF
0	1	1	0b 0000 1011 (0x0B)	OFF	ON	ON
1	0	0	0b 0000 1100 (0x0C)	ON	OFF	OFF
1	0	1	0b 0000 1101 (0x0D)	ON	OFF	ON
1	1	0	0b 0000 1110 (0x0E)	ON	ON	OFF
1	1	1	0b 0000 1111 (0x0F)	ON	ON	ON

(5) Cursor Or Display Shift

DD RAM의 내용을 변경하지 않고 화면 또는 커서(어드레스)를 오른쪽 또는 왼쪽으로 시프트(shfit)한다.

Rs	R/nW	DB6	DB5	DB4	DB3	DB2	DB1	DB0
0	0	0	0	1	S/C	R/L	X	X

S/C => Shift/Cursor R/L => Right/Left

S/C	R/L	Value	기능
0	0	0b 0001 00XX (0x10)	커서(어드레스) 선택, 커서(어드레스) 좌측 이동
0	1	0b 0001 01XX (0x14)	커서(어드레스) 선택, 커서(어드레스) 우측 이동
1	0	0b 0001 10XX (0x18)	화면 선택, 화면 좌측 이동
1	1	0b 0001 11XX (0x1C)	화면 선택, 화면 우측 이동

(6) Function Set

MCU와 LCD 모듈 사이에 인터페이스 설정을 한다.

데이터 길이, 화면 표시 행수, 문자 폰트 크기를 설정한다.

Rs	R/nW	DB6	DB5	DB4	DB3	DB2	DB1	DB0
0	0	0	1	DL	N	F	X	X

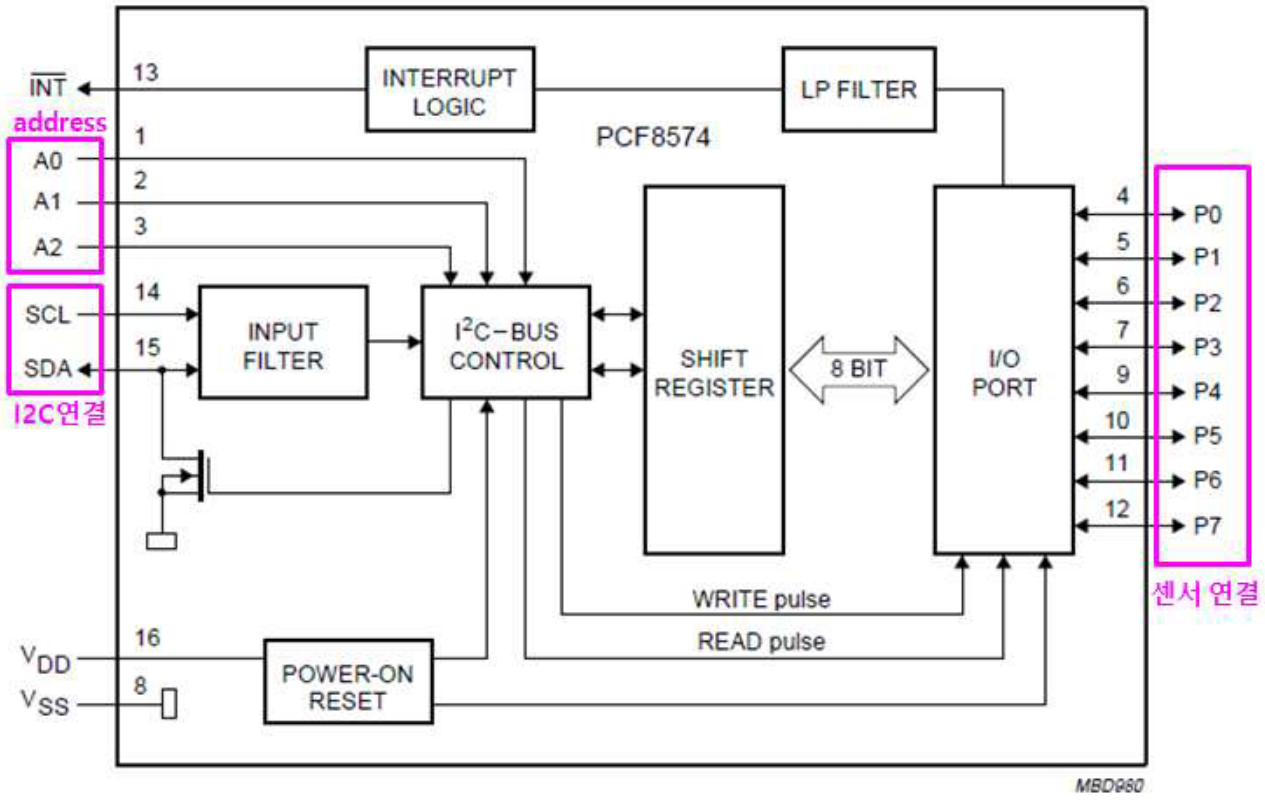
DL => Data Length N => New Line F => Font

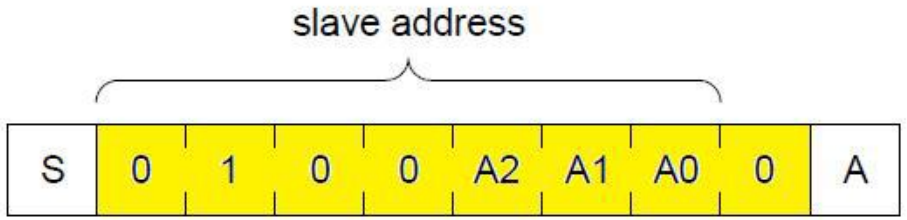
DL	N	F	기능		
			데이터 길이	행 수	폰트 크기
0	0	0	4bit	1행	5 * 8
0	0	1	4bit	1행	5 * 11
0	1	0	4bit	2행	5 * 8
0	1	1	4bit	2행	5 * 11
1	0	0	8bit	1행	5 * 8
1	0	1	8bit	1행	5 * 11
1	1	0	8bit	2행	5 * 8
1	1	1	8bit	2행	5 * 11

LCD(4/8)

PCF8574 IC

- 2개의 핀을 사용하는 I2C 방식의 집적회로





Pin Connectivity			Address of PCF8547								7bit
A2	A1	A0	A6	A5	A4	A3	A2	A1	A0	0	Address
L	L	L	0	1	0	0	0	0	0	-	20h
L	L	H	0	1	0	0	0	0	1	-	21h
L	H	L	0	1	0	0	0	1	0	-	22h
L	H	H	0	1	0	0	0	1	1	-	23h
H	L	L	0	1	0	0	1	0	0	-	24h
H	L	H	0	1	0	0	1	0	1	-	25h
H	H	L	0	1	0	0	1	1	0	-	26h
H	H	H	0	1	0	0	1	1	1	-	27h

LCD(5/8)

라이브러리 함수

int lcdInit (const int rows, const int cols, const int bits, const int rs, const int strb, const int d0, const int d1, const int d2, const int d3, const int d4, const int d5, const int d6, const int d7)		
위 치	wiringPi/devLib/lcd.h	
기 능	I2C 통신할 슬레이브를 식별할 주소 설정	
매개변수	const int rows	LCD의 라인 수
	const int cols	LCD의 칸 수
	const int bits	LCD의 데이터 제어 비트 수 (8bit / 4bit)
	const int rs	데이터 레지스터, 명령어 레지스터 선택 핀
	const int strb	LCD에 데이터를 쓰겠다는 신호 핀
	const int d0	데이터 핀0
	const int d1	데이터 핀1
	const int d2	데이터 핀2
	const int d3	데이터 핀3
	const int d4	데이터 핀4
	const int d5	데이터 핀5
	const int d6	데이터 핀6
	const int d7	데이터 핀7
리턴 값	int	

int pcf8574Setup(const int pinBase, const int i2cAddress)		
위 치	wiringPi/wiringPi/pcf8574.h	
기 능	PCF8574 IC와 LCD 연결 설정 및 초기화	
매개변수	const int pinBase	64이상의 8개 핀의 시작 주소
리턴 값	const int i2cAddress	I2C통신 주소

void lcdHome(const int fd)		
위 치	wiringPi/devLib/lcd.h	
기 능	커서를 0번 칸, 0번 줄로 이동	
매개변수	const int fd	LCD 커서 제어 핸들
리턴 값	void	없음

void lcdClear(const int fd)		
위 치	wiringPi/devLib/lcd.h	
기 능	LCD화면 지우기	
매개변수	const int fd	LCD 커서 제어 핸들
리턴 값	void	없음

LCD(6/8)

라이브러리 함수

void lcdDisplay(const int fd, int state)		
위 치	wiringPi/devLib/lcd.h	
기 능	LCD를 화면 On/Off 여부 설정	
매개변수	const int fd	LCD 커서 제어 핸들
	int state	화면 On/Off 여부
리턴 값	void	없음

void lcdCursor(const int fd, int state)		
위 치	wiringPi/devLib/lcd.h	
기 능	LCD에 커서 표시 여부 설정	
매개변수	const int fd	LCD 커서 제어 핸들
	int state	커서 표시 여부
리턴 값	void	없음

void lcdCursorBlink(const int fd, int state)		
위 치	wiringPi/devLib/lcd.h	
기 능	LCD에 커서 깜빡임 여부 설정	
매개변수	const int fd	LCD 커서 제어 핸들
	int state	커서 깜빡임 여부
리턴 값	void	없음

void lcdSendCommand(const int fd, unsigned char cmd)		
위 치	wiringPi/devLib/lcd.h	
기 능	LCD에 명령어 전송	
매개변수	const int fd	LCD 커서 제어 핸들
	unsigned char cmd	LCD에 보낼 커맨드
리턴 값	void	없음

void lcdPosition(const int fd, int x, int y)		
위 치	wiringPi/devLib/lcd.h	
기 능	지정된 위치에 커서 위치	
매개변수	const int fd	LCD 커서 제어 핸들
	int x	커서가 놓일 x좌표
	int y	커서가 놓일 y좌료
리턴 값	void	없음

void lcdPutchar(const int fd, unsigned char data)		
위 치	wiringPi/devLib/lcd.h	
기 능	LCD에 문자 출력	
매개변수	const int fd	LCD 커서 제어 핸들
	unsigned char data	출력할 ASCII 문자
리턴 값	void	없음

LCD(7/8)

라이브러리 함수

void lcdPuts(const int fd, const char *string)		
위 치	wiringPi/devLib/lcd.h	
기 능	LCD에 문자열 출력	
매개변수	const int fd	LCD 커서 제어 핸들
	const char *string	출력할 ASCII 문자열
리턴 값	void	없음

void lcdPrintf(const int fd, const char *message, . . .)		
위 치	wiringPi/devLib/lcd.h	
기 능	LCD에 형식지정 문자열 출력	
매개변수	const int fd	LCD 커서 제어 핸들
	const char *message	출력할 형식지정 문자열
리턴 값	void	없음

void lcdCharDef(const int fd, int x, unsigned char data[8])		
위 치	wiringPi/devLib/lcd.h	
기 능	8개 문자를 사용자 정의	
매개변수	const int fd	LCD 커서 제어 핸들
	int index	커서가 놓일 x좌표
	unsigned char data[8]	사용자 정의 8개 문자
리턴 값	void	없음

LCD(8/8)

```c
#include <wiringPi.h>
#include <pcf8574.h>
#include <lcd.h>
#include <stdio.h>

#define BASE_ADDR 100
#define RS_PIN BASE_ADDR //- 100
#define RW_PIN BASE_ADDR+1 //- 101
#define EN_PIN BASE_ADDR+2 //- 102
#define BL_PIN BASE_ADDR+3 //- 103
#define D4_PIN BASE_ADDR+4 //- 104
#define D5_PIN BASE_ADDR+5 //- 105
#define D6_PIN BASE_ADDR+6 //- 106
#define D7_PIN BASE_ADDR+7 //- 107

int main (void){
  //- wiringPi 초기화
  if( wiringPiSetup() == -1){
    printf("Setup Fail ~!!");
    return -1;
  }
```

```c
//- I2C 통신기반 LCD 슬레이브 설정
  pcf8574Setup(RS_PIN, 0x27); //- P0핀 - LCD의 RS 연결, 주소 지정
  //- P1핀 - LCD의 RW 연결, 동작모드 설정
  pinMode (RW_PIN, OUTPUT);
  digitalWrite (RW_PIN, LOW);
  //- P3핀 - LCD의 Backlight 연결
  pinMode (BL_PIN, OUTPUT);
  digitalWrite (BL_PIN, HIGH); //- Backlight ON
  //- LCD 제어 핸들
  int fd = lcdInit (2, 16, 4, RS_PIN, EN_PIN, D4_PIN, D5_PIN, D6_PIN,
  D7_PIN, 0,0,0,0 );
  while(1){
    lcdClear(fd); //- LCD화면 지우기
    delay(500);
    lcdPosition (fd, 2, 0); // 윗줄 2번째 칸
    lcdPuts (fd, " Hello, World! " ); //- 문자열 출력
    lcdPosition (fd, 2, 1); // 아랫줄 2번째 칸
    lcdPuts (fd, "* Good Luck *"); //- 문자열 출력
    delay(2000);
  }
  lcdClear(fd); //- LCD화면 지우기
  delay(500);
  return 0;
}
```

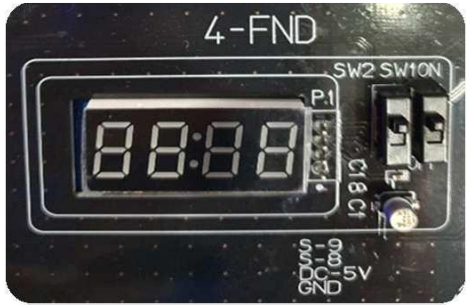
컴파일

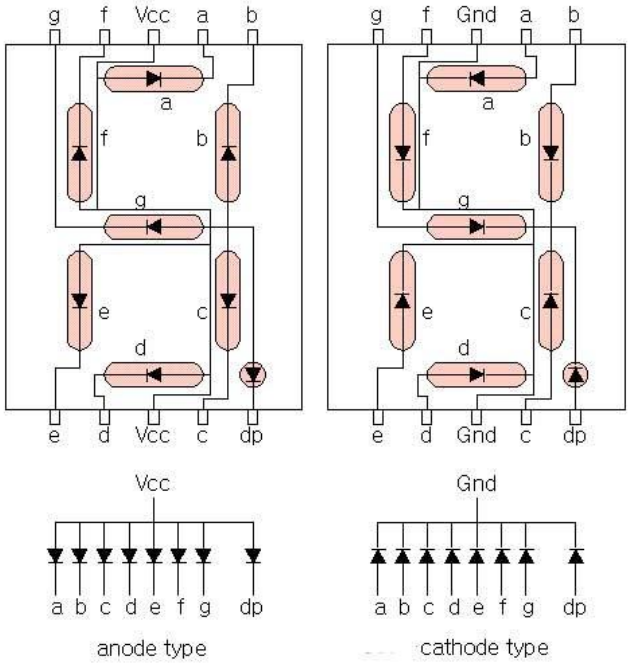
ex) gcc -o [**실행파일명**] [**코드파일**] -lwiringPi -lwiringPiDev

4-FND(1/6)

FND(Flexible Numeric Display)

- 7-segment: 7개의 LED 소자를 8자 모양으로 배치하여 0~9까지 숫자를 표현하는 장치
- VCC와 GND에 연결된 방식에 따라 캐소드(Cathode)와 애노드(Anode)로 구분

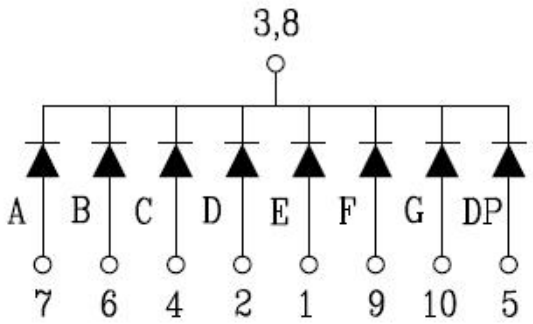


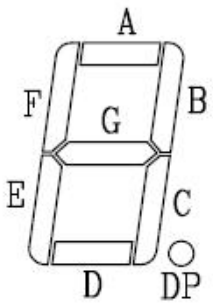


4-FND(2/6)

7-Segment의 원리

- 캐소드 타입
- 각 LED 소자의 공통 단자는 GND에 연결되어 있다.
- 각 LED를 켜기 위해선 HIGH, 끄기 위해선 LOW를 인가한다.





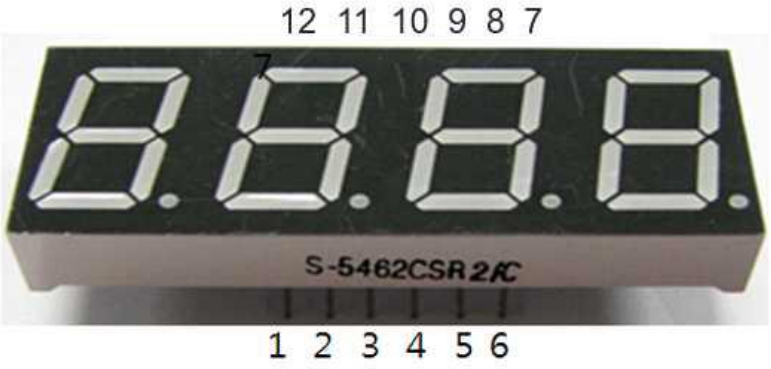
문자 숫자	a (PC5)	b (PC4)	c (PC2)	d (PC1)	e (PC0)	f (PC6)	g (PC7)	dp (PC3)
0	1	1	1	1	1	1	0	0
1	0	1	1	0	0	0	0	0
2	1	1	0	1	1	0	1	0
3	1	1	1	1	0	0	1	0
4	0	1	1	0	0	1	1	0
5	1	0	1	1	0	1	1	0
6	0	0	1	1	1	1	1	0
7	1	1	1	0	0	0	0	0
8	1	1	1	1	1	1	1	0
9	1	1	1	0	0	1	1	0
a	1	1	1	1	1	0	1	0
b	0	1	0	1	1	1	1	0
c	1	0	0	1	1	1	0	0
d	0	1	1	1	1	0	1	0
e	1	0	0	1	1	1	1	0
f	1	0	0	0	1	1	1	0

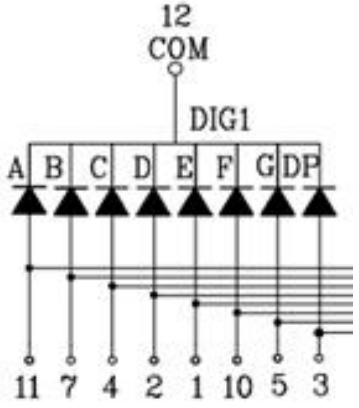
7-Segment 문자 & 숫자 표시 값

4-FND(3/6)

4-Digit 7-Segment

- 7개의 LED 핀과 더불어 각 자리를 제어하는 핀이 추가되었다.



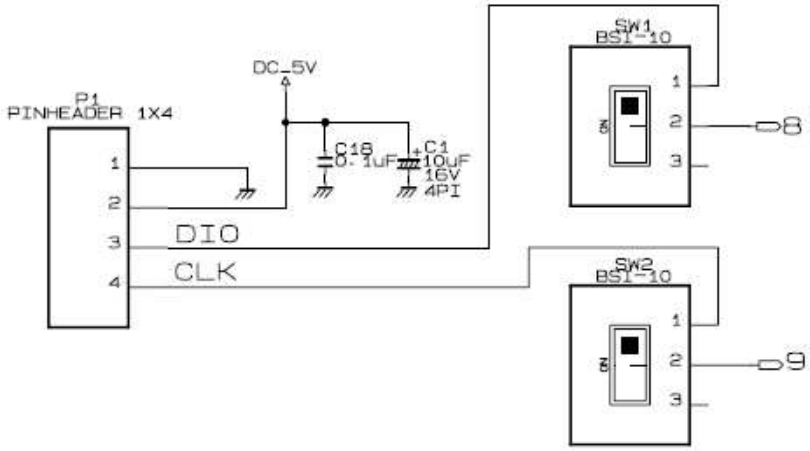


모듈 핀	Segment
1	E segment
2	D segment
3	DP segment
4	C segment
5	G segment
6	**COMMENT**
7	B segment
8	**COMMENT**
9	**COMMENT**
10	F segment
11	A segment
12	**COMMENT**

4-digit 7 seg의 핀 맵

4-FND(4/6)

4-FND의 연결회로




```
+-----+-----+---------+------+---+---Pi 3B+--+---+------+---------+-----+-----+
| BCM | wPi |   Name  | Mode | V | Physical | V | Mode | Name    | wPi | BCM |
+-----+-----+---------+------+---+----++----+---+------+---------+-----+-----+
|     |     |    3.3v |      |   |  1 || 2  |   |      | 5v      |     |     |
|   2 |   8 |   SDA.1 | ALT0 | 1 |  3 || 4  |   |      | 5v      |     |     |
|   3 |   9 |   SCL.1 | ALT0 | 1 |  5 || 6  |   |      | 0v      |     |     |
|   4 |   7 | GPIO. 7 |   IN | 1 |  7 || 8  | 0 | IN   | TxD     | 15  | 14  |
|     |     |      0v |      |   |  9 || 10 | 1 | IN   | RxD     | 16  | 15  |
|  17 |   0 | GPIO. 0 |   IN | 1 | 11 || 12 | 1 | IN   | GPIO. 1 | 1   | 18  |
|  27 |   2 | GPIO. 2 |   IN | 0 | 13 || 14 |   |      | 0v      |     |     |
```

4-FND(5/6)

TM1637 라이브러리

void TM1637_init(uint8_t Clk, uint8_t Data)		
위 치	TM1637.h	
기 능	pwm 전용 핀에 pwm파형 출력	
매개변수	uint8_t Clk	클럭 핀 지정
	uint8_t Data	데이터 송수신 핀 지정
리턴 값	void	없음

void TM1637_writeByte(int8_t wr_data)		
위 치	TM1637.h	
기 능	software pwm 핀에 PWM 출력	
매개변수	uint8_t Data	8비트 데이터를 TM1637에 보내기
리턴 값	void	없음

void TM1637_start(void)		
위 치	TM1637.h	
기 능	일반 핀에 pwm파형 출력을 정지	
매개변수	void	없음
리턴 값	void	없음

void TM1637_stop(void)		
위 치	TM1637.h	
기 능	일반 핀에 pwm파형 출력을 정지	
매개변수	void	없음
리턴 값	void	없음

void TM1637_set(uint8_t brightness,uint8_t SetData,uint8_t SetAddr)		
위 치	TM1637.h	
기 능	일반 핀에 pwm파형 출력을 정지	
매개변수	uint8_t brightness	LED 밝기 설정 #define BRIGHT_DARKEST 0 #define BRIGHT_TYPICAL 2 #define BRIGHTEST 7
	uint8_t SetData	데이터 읽기 또는 쓰기 모드 설정 주소 지정 모드, 동작 모드 설정 0x40=>01000000 : 쓰기, 자동어드레싱, 일반 모드
	uint8_t SetAddr	디스플레이 레지스터 주소 설정
리턴 값	void	없음

4-FND(6/6)

```c
#include <stdio.h>
#include <wiringPi.h>
#include "TM1637.h"
#define CLK_PIN 9
#define DIO_PIN 8
int main(void){
    int8_t NumTab[] = {0,1,2,3,4,5,6,7,8,9,10,11,12,13,14,15};
    int8_t ListDisp[4];
    unsigned char i = 0, count = 0;
    delay(150);
    //- wiringPi 초기화
    if( wiringPiSetup() == -1){
        printf("Setup Fail ~!!");
        return -1;
    }
    //- 2개 핀 동작 모드 설정
    pinMode(CLK_PIN,INPUT);
    pinMode(DIO_PIN,INPUT);
    delay(200);
    TM1637_init(CLK_PIN,DIO_PIN);
    TM1637_set( BRIGHTEST, 0x40, 0xc0 );
```

```c
  while(1){
        unsigned char BitSelect = 0;
        i = count;
        count ++;
        if(count == sizeof(NumTab)) count = 0;
        for(BitSelect = 0;BitSelect < 4;BitSelect ++){
            ListDisp[BitSelect] = NumTab[i];
            i++;
            if(i == sizeof(NumTab)) i = 0;
        }
        TM1637_display(0,ListDisp[0]);
        TM1637_display(1,ListDisp[1]);
        TM1637_display(2,ListDisp[2]);
        TM1637_display(3,ListDisp[3]);
        delay(500);
    }
    return 0;
}
```

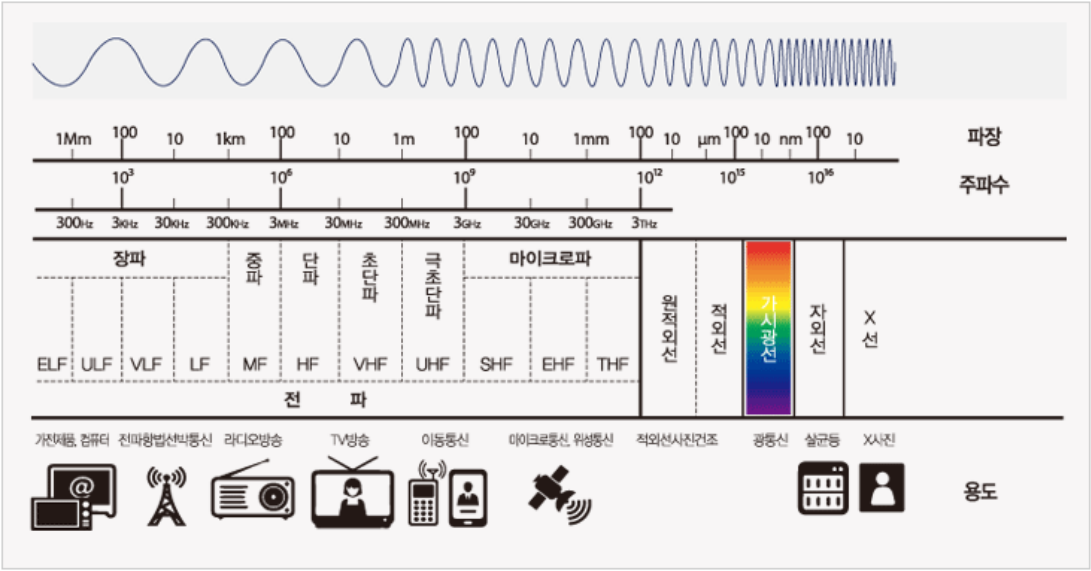
컴파일

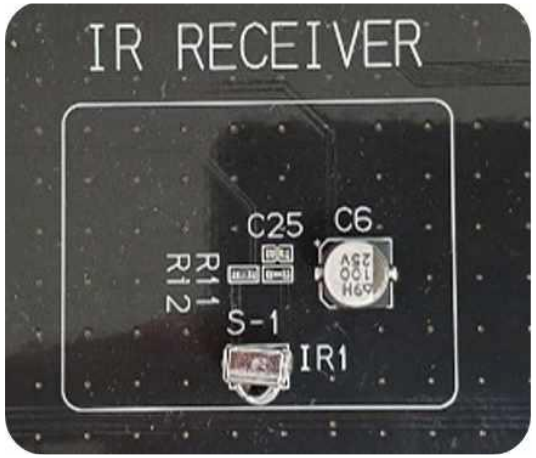
ex) gcc -o [**실행파일명**] [**코드파일**] TM1637.c -lwiringPi -fcommon

IR(1/4)

IR Receiver

- 적외선 리모컨 또는 적외선 송신기에서 보내는 적외선 신호를 받는 장치
- 자연광의 IR의 간섭을 피하기 위해 38KHz 주파수를 이용
- 통상적으로 NEC 포멧을 이용한다.

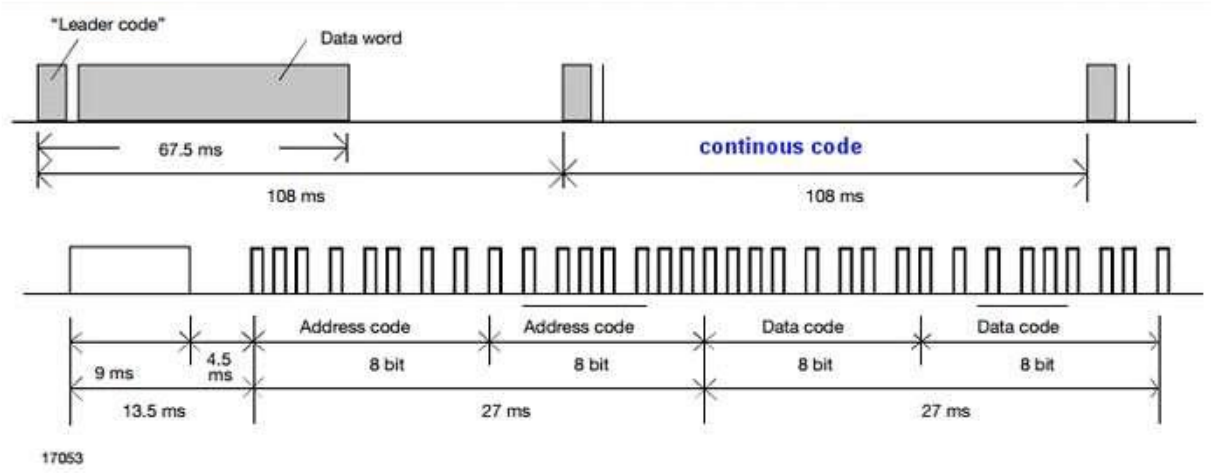




IR(2/4)

NEC

- Leader Code(9ms) + Space(4.5ms) + Data Word(54ms)
- 32bit = Address code(8bit) + Address Code1의 보수(8bit) + Data code(8bit) + Data code 1의 보수(8bit)



IR(3/4)

PIGPIO 설치 (설치가 되어 있음. 필요한 경우에만 수행)

- 아래의 절차를 순차적으로 수행
- wget https://github.com/joan2937/pigpio/archive/master.zip
- unzip master.zip
- cd pigpio-master
- make
- sudo make install
- 출처: https://abyz.me.uk/rpi/pigpio/download.html

IR(4/4)

```c
#include "irpigpio.h"
#include <stdio.h>
#include <wiringPi.h>
#define IR_PIN 18
#define LED 27
int deviceSetup(){
    if (gpioInitialise()<0 || wiringPiSetup()<0){
     return -1 ;
    }
    /* IR pin as input */
    gpioSetMode(IR_PIN, PI_INPUT);
    /* 5ms max gap after last pulse */
    gpioSetWatchdog(IR_PIN, 5);
    /* monitor IR level changes */
    gpioSetAlertFunc(IR_PIN, alert);
}
int main(){
    long value;
    if(deviceSetup()<0)
        return 1;
    pinMode(LED, OUTPUT);
    while(1){
        value = readIR();
        printf("IR value %x\n", value);
    }
    gpioTerminate();
}
```

```c
#include "irpigpio.h"
#include <stdio.h>
#include <wiringPi.h>
#define IR_PIN 18
#define LED 27
#define BUTTON_0 0x102a4f46
#define BUTTON_1 0x8534077e
int deviceSetup(){

            // 옆의 예제와 동일

}
int main(){
    long value;
    if(deviceSetup()<0) return 1;
    pinMode(LED, OUTPUT);
    while(1){
        value = readIR();
        switch(value){
            case BUTTON_0:printf("Button 0 pressed, LED On\n");
            digitalWrite(LED, HIGH);
            break;
            case BUTTON_1:printf("Button 1 pressed, LED Off\n");
            digitalWrite(LED, LOW);
            break;
        }
    }
    gpioTerminate();
}
```

컴파일

ex) gcc -o [**실행파일명**] [**코드파일**] irpigpio.c -lpthread -lwiringPi -lpigpio

실행

sudo ./[**실행파일명**]

과제 9

FND에 프로그램의 실행결과를 출력한다.

- 프로그램을 실행하면 시스템은 2가지의 기능을 갖는다.
- 키보드의 q 를 누르면 두개의 양의 정수를 입력 받는 문구가 출력된다.
- scanf를 이용해 두 개의 양의 정수를 입력했을 때 두 수의 합을 출력한다.
- 값이 출력된 이후에 키보드의 e를 누르면 해당 숫자가 1씩 줄어들고, 0이 되면 LED가 1초 동안 켜진다.
- q를 이용해 양의 정수를 입력 받지 않은 상태에서 e를 누르면 아무일도 일어나지 않는다.

과제 10

IR센서와 리모컨을 사용한 프로그램의 실행결과를 LCD에 출력한다.

- 초기의 LCD에는 아무것도 출력되지 않는다.
- 리모컨의 1번을 누르면 LCD에 ‘Open the door’ 가 출력되고 1초 뒤 사라진다.
- 리모컨의 2번을 누르면 LCD에 ‘Close the door’ 가 출력되고 1초 뒤 사라진다.
- 문이 열릴 때는 상승하는 음계가 수동 부저에서 출력된다. ex) 도 미 솔 ~
- 문이 닫힐 때는 하강하는 음계가 수동 부저에서 출력된다. ex) 솔 미 도 ~
- 문이 열린 상태에서 1번을 누르거나, 닫힌 상태에서 2번을 누르면 아무 일도 일어나지 않는다.
- 리모컨의 2번을 눌렀을 때, 신호 값이 불안정하다면 다른 버튼을 사용한다.

고생하셨습니다

Thank you!